Diagrama de Clases UML - Sistema de Gestión Fast Food (Backend REST + Frontend web)

VISTA (Frontend web · JSP)
«Controller» Controladores web
AuthController · InicioController · EmpleadoController
IngredienteController · ProductoController · PedidoController
+atienden cada pantalla JSP y arman el Model
«final» Sesion
+iniciar(sesion: HttpSession, cred: Credenciales, usuario: UsuarioActualDTO): void
+estaIniciada(sesion: HttpSession): boolean
+credenciales(sesion: HttpSession): Credenciales
+usuario(sesion: HttpSession): UsuarioActualDTO
+tieneRol(sesion: HttpSession, rol: String): boolean
+cerrar(sesion: HttpSession): void
«record» Credenciales
+usuario: String
+password: String
«ControllerAdvice» ErroresAdvice
+manejarBackend(e: BackendException, sesion: HttpSession, model: Model): String
maneja
BackendException
-status: int
+isNoAutenticado(): boolean
+isSinConexion(): boolean
lanza
«Component» SesionInterceptor
+preHandle(request, response, handler): boolean
«Component» BackendClient
-restTemplate: RestTemplate
-backendUrl: String
+get(ruta: String, tipo: Class<T>, cred: Credenciales): T
+getLista(ruta: String, tipo: Class<T[]>, cred: Credenciales): List<T>
+post(ruta: String, body: Object, tipo: Class<T>, cred: Credenciales): T
+put(ruta: String, body: Object, tipo: Class<T>, cred: Credenciales): T
+putSinRespuesta(ruta: String, body: Object, cred: Credenciales): void
+delete(ruta: String, cred: Credenciales): void
-llamar(metodo, ruta, body, tipo, cred): T
HTTP/JSON
HTTP/JSON
HTTP/JSON
HTTP/JSON
HTTP/JSON
CONTROLADOR (API REST)
«RestController» PedidoController
+listarPedidos(): List<PedidoDTO>
+listarPendientes(): List<PedidoDetalleDTO>
+reporteVentas(desde: LocalDate, hasta: LocalDate, idVendedor: String): ReporteVentasDTO
+buscarPedido(id: String): PedidoDetalleDTO
+registrarPedido(dto: PedidoRequestDTO, auth: Authentication): PedidoDetalleDTO
+despacharPedido(id: String, auth: Authentication): PedidoDetalleDTO
+eliminarPedido(id: String): void
«RestController» ProductoController
+listarProductos(): List<ProductoDTO>
+listarDisponibles(): List<ProductoDTO>
+buscarProducto(id: String): ProductoDTO
+crearProducto(dto: ProductoRequestDTO): ProductoDTO
+modificarProducto(id: String, dto: ProductoRequestDTO): ProductoDTO
+darDeBajaProducto(id: String): void
+verReceta(id: String): List<ProductoIngredienteDTO>
+agregarIngrediente(id: String, dto: RecetaItemDTO): ProductoIngredienteDTO
+quitarIngrediente(id: String, idIngrediente: String): void
«RestController» AuthController
+usuarioActual(authentication: Authentication): UsuarioActualDTO
+cambiarPassword(dto: CambioPasswordDTO, authentication: Authentication): void
«RestController» IngredienteController
+listarIngredientes(): List<IngredienteDTO>
+buscarIngrediente(id: String): IngredienteDTO
+crearIngrediente(dto: IngredienteRequestDTO): IngredienteDTO
+modificarIngrediente(id: String, dto: IngredienteRequestDTO): IngredienteDTO
+darDeBajaIngrediente(id: String): void
«RestController» EmpleadoController
+listarEmpleados(auth: Authentication): List<EmpleadoDTO>
+buscarEmpleado(id: String, auth: Authentication): EmpleadoDTO
+registrarEmpleado(dto: EmpleadoRequestDTO, auth: Authentication): EmpleadoDTO
+modificarEmpleado(id: String, dto: EmpleadoRequestDTO, auth: Authentication): EmpleadoDTO
+cambiarRol(id: String, dto: CambioRolDTO, auth: Authentication): EmpleadoDTO
+darDeBajaEmpleado(id: String, auth: Authentication): void
«RestController» PruebaController
+prueba(): String
SERVICIO (Reglas de negocio)
«Service» PedidoService
+listarTodos(): List<Pedido>
+listarPendientes(): List<Pedido>
+buscar(id: String): Pedido
+registrar(usuarioVendedor: String, datos: PedidoRequestDTO): Pedido
+despachar(usuarioCocinero: String, idPedido: String): Pedido
+eliminar(idPedido: String): void
+pedidosParaReporte(desde: LocalDate, hasta: LocalDate, idVendedor: String): List<Pedido>
«Service» ProductoService
+listarActivos(): List<Producto>
+listarDisponibles(): List<Producto>
+buscar(id: String): Producto
+registrar(descripcion: String, precio: Double): Producto
+modificar(id: String, descripcion: String, precio: Double): Producto
+darDeBaja(id: String): void
+verReceta(idProducto: String): List<ProductoIngrediente>
+agregarIngrediente(idProducto: String, idIngrediente: String, obligatorio: boolean): ProductoIngrediente
+quitarIngrediente(idProducto: String, idIngrediente: String): void
«Service» IngredienteService
+listarActivos(): List<Ingrediente>
+buscar(id: String): Ingrediente
+registrar(descripcion: String, precio: Double): Ingrediente
+modificar(id: String, descripcion: String, precio: Double): Ingrediente
+darDeBaja(id: String): void
«utilitaria» GeneradorId
+siguiente(idsExistentes: Collection<String>, prefijo: String, digitos: int): String
«Service» EmpleadoService
+listar(usuarioActor: String): List<Empleado>
+buscar(usuarioActor: String, id: String): Empleado
+registrar(usuarioActor: String, datos: EmpleadoRequestDTO): Empleado
+modificar(usuarioActor: String, id: String, datos: EmpleadoRequestDTO): Empleado
+cambiarRol(usuarioActor: String, id: String, rolTexto: String): Empleado
+cambiarPasswordPropia(usuarioActor: String, passwordActual: String, passwordNueva: String): void
+darDeBaja(usuarioActor: String, id: String): void
+crearSysAdminInicialSiNoHayEmpleados(): void
lanza
REPOSITORIO (Spring Data JPA)
JpaRepository T, ID
+findAll(): List<T>
+findById(id: ID): Optional<T>
+existsById(id: ID): boolean
+save(entidad: T): T
+delete(entidad: T): void
+count(): long
ProductoRepository
+existsByDescripcionIgnoreCase(descripcion: String): boolean
+existsByDescripcionIgnoreCaseAndIDNot(descripcion: String, id: String): boolean
+findByActivoTrue(): List<Producto>
+findByActivoTrueAndDisponibleTrue(): List<Producto>
IngredienteRepository
+existsByDescripcionIgnoreCase(descripcion: String): boolean
+existsByDescripcionIgnoreCaseAndIDNot(descripcion: String, id: String): boolean
+findByActivoTrue(): List<Ingrediente>
EmpleadoRepository
+findByUsuario(usuario: String): Optional<Empleado>
+existsByDNI(dni: String): boolean
+existsByUsuario(usuario: String): boolean
+existsByUsuarioAndIDNot(usuario: String, id: String): boolean
+findByRolIn(roles: Collection<? extends Rol>): List<Empleado>
+countByRolAndActivoTrue(rol: Rol): long
PedidoRepository
+findByEstadoOrderByFechaAscIDAsc(estado: EstadoPedido): List<Pedido>
+findAllByOrderByFechaAscIDAsc(): List<Pedido>
+findAllIds(): List<String>
+buscarParaReporte(estado: EstadoPedido, desde: LocalDate, hasta: LocalDate, idVendedor: String): List<Pedido>
ProductoIngredienteRepository
+findByProducto_ID(idProducto: String): List<ProductoIngrediente>
+existsByProducto_IDAndObligatorioTrue(idProducto: String): boolean
+existsByIngrediente_IDAndProducto_ActivoTrue(idIngrediente: String): boolean
ItemIngredienteRepository
+existsByIngrediente_IDAndItem_Pedido_Estado(idIngrediente: String, estado: EstadoPedido): boolean
+findAllIds(): List<String>
ItemRepository
+existsByProducto_IDAndPedido_Estado(productoId: String, estado: EstadoPedido): boolean
+findAllIds(): List<String>
EXCEPCIONES
RuntimeException
«RestControllerAdvice» GlobalExceptionHandler
+manejarNoEncontrado(ex: RecursoNoEncontradoException): ProblemDetail
+manejarDatoInvalido(ex: DatoInvalidoException): ProblemDetail
+manejarReglaNegocio(ex: ReglaNegocioException): ProblemDetail
+manejarOperacionNoPermitida(ex: OperacionNoPermitidaException): ProblemDetail
+manejarJsonInvalido(ex: HttpMessageNotReadableException): ProblemDetail
maneja (404 / 400 / 409 / 403)
NegocioException
OperacionNoPermitidaException
ReglaNegocioException
DatoInvalidoException
RecursoNoEncontradoException
SEGURIDAD Y CONFIGURACIÓN
ApplicationRunner
+run(args: ApplicationArguments): void
«Component» InicializadorDatos
+run(args: ApplicationArguments): void
UserDetailsService
+loadUserByUsername(usuario: String): UserDetails
«Service» EmpleadoUserDetailsService
+loadUserByUsername(usuario: String): UserDetails
PasswordEncoder
+encode(raw: CharSequence): String
+matches(raw: CharSequence, codificada: String): boolean
TextoPlanoLegacyEncoder
+encode(rawPassword: CharSequence): String
+matches(rawPassword: CharSequence, storedPassword: String): boolean
«Configuration» SecurityConfig
-GERENTE: String
-VENDEDOR: String
-COCINERO: String
-SYS_ADMIN: String
+securityFilterChain(http: HttpSecurity): SecurityFilterChain
crea
MODELO (Model)
Entidades del sistema
«Entity» ProductoIngrediente
-producto: Producto
-ingrediente: Ingrediente
-obligatorio: boolean
«Entity» Producto
-ID: String
-descripcion: String
-precio: double
-disponible: boolean
-activo: boolean
receta de
producto
lleva
0..*
1
«Entity» Ingrediente
-ID: String
-descripcion: String
-precio: double
-activo: boolean
«Entity» Item
-ID: String
-pedido: Pedido
-producto: Producto
-descripcionProducto: String
-cantidad: int
-precioVenta: double
-modificaciones: List<ItemIngrediente>
+agregarModificacion(modificacion: ItemIngrediente): void
+calcularCostoAgregadosPorUnidad(): double
+calcularSubtotal(): double
«Entity» Pedido
-ID: String
-fecha: LocalDate
-estado: EstadoPedido
-empleadoVendedor: Empleado
-empleadoCocinero: Empleado
-items: List<Item>
+agregarItem(item: Item): void
+despachar(cocinero: Empleado): void
+calcularTotal(): double
items (composición)
1..*
vendedor
cocinero
0..1
«Entity» Empleado
-activo: boolean
«Entity» Persona
-ID: String
-DNI: String
-nombre: String
-apellido: String
-usuario: String
-contraseña: String
-rol: Rol
«Entity» ItemIngrediente
-ID: String
-item: Item
-ingrediente: Ingrediente
-tipoModificacion: TipoModificacion
-cantidad: int
-precioUnitario: double
+calcularCostoPorUnidad(): double
modificaciones (composición)
ingrediente
«IdClass»
Serializable
«IdClass» ProductoIngredienteId
-producto: String
-ingrediente: String
+equals(o: Object): boolean
+hashCode(): int
tipoModificacion
estado
rol
Enumeraciones
TipoModificacion
AGREGADO
QUITADO
EstadoPedido
PENDIENTE
DESPACHADO
Rol
VENDEDOR
COCINERO
GERENTE
SYS_ADMIN
Objetos de Transferencia (DTOs)
EmpleadoRequestDTO
PedidoDetalleDTO
ProductoRequestDTO
EmpleadoDTO
PedidoDTO
ProductoIngredienteDTO
UsuarioActualDTO
CambioRolDTO
IngredienteRequestDTO
ProductoDTO
ReporteVentasDTO
CambioPasswordDTO
IngredienteDTO
PedidoRequestDTO
RecetaItemDTO